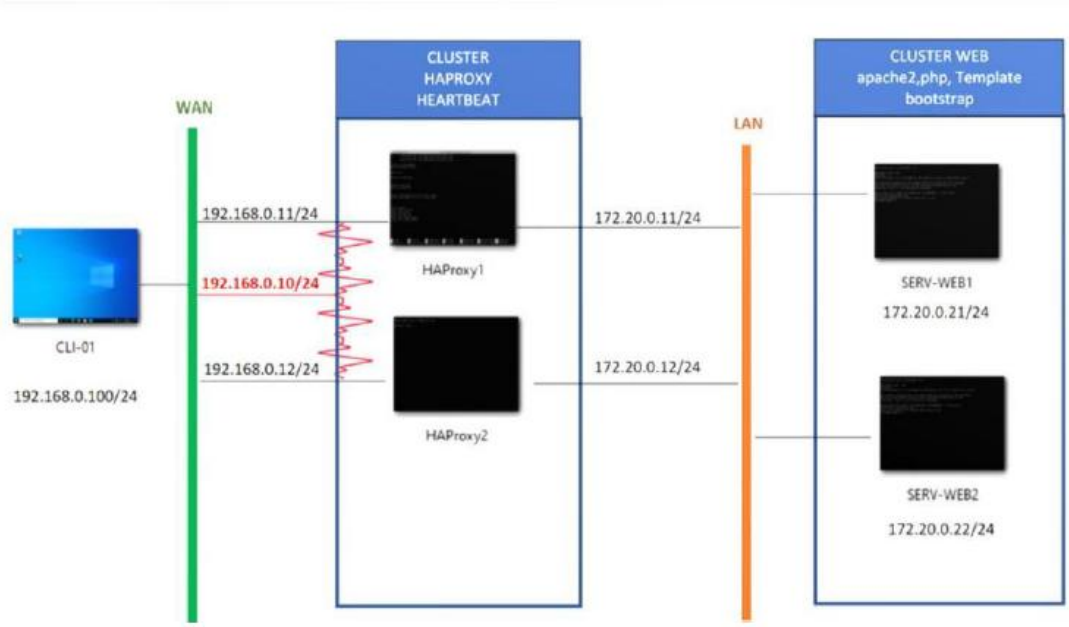
BTS SIO – 1RE ANNEE

Projet Haute Disponibilité

Mise en place d'un cluster web haute disponibilité

HAProxy + Heartbeat



Réalisé par	Julie Durand
Formation	BTS SIO – 1re année
Environnement	VMware Workstation
Date de rendu	13 avril 2026

Sommaire

1. Introduction 4
1.1 Contexte 4
1.2 Objectifs pédagogiques 4
1.3 Enjeux de la haute disponibilité 4
Enjeu économique 4
Enjeu d'image et de confiance 4
Enjeu technique : résilience et redondance 5
Enjeu de sécurité et d'intégrité 5
2. Présentation de l'architecture 6
2.1 Schéma d'architecture 6
2.2 Tableau d'adressage 6
2.3 Plan d'adressage réseau 7
3. Réalisation étape par étape 8
Étape 1 – Préparation de l'environnement VMware 8
Création des réseaux virtuels WAN et LAN 8
Création des cinq machines virtuelles 8
Configuration des interfaces réseau 8
Vérification de la connectivité 9
Étape 2 – Déploiement des serveurs web 10
Installation d'Apache2 et PHP 10
Déploiement du site « The Grill » 10
Modification du DocumentRoot d'Apache2 10
Personnalisation de la page d'accueil de chaque serveur 11
Activation de Bootstrap via un CDN 11
Vérification de l'accès depuis les nœuds HAProxy 12
Étape 3 – Configuration du cluster HAProxy 12
Installation de HAProxy 12
Configuration identique sur les deux nœuds 12
Activation de la page de statistiques 13
Test du load balancing depuis CLI-01 13
Étape 4 – Configuration du cluster Heartbeat 13
Installation de Heartbeat 13
Fichier ha.cf 14
Fichier haresources 14
Fichier authkeys 14
Définition de la VIP comme ressource Heartbeat 14

Vérification de la VIP sur le nœud maître 15
Étape 5 – Tests de bascule et validation 15
Test 1 – Répartition de charge (round-robin) 16
Test 2 – Panne du nœud HAProxy1 (failover) 16
Test 3 – Panne d'un serveur backend (health-check) 17
Test 4 – Retour du nœud primaire (failback) 18
4. Bilan 20
4.1 Synthèse des tests 20
4.2 Conclusion 20

1. Introduction

1.1 Contexte

Dans le cadre de ma formation en BTS SIO, j'ai étudié deux outils fondamentaux de la haute disponibilité sous Linux :

- **HAProxy**, pour la répartition de charge (load balancing) entre plusieurs serveurs web ;
- **Heartbeat**, pour assurer la continuité de service grâce à une adresse IP virtuelle (VIP) partagée entre deux nœuds.

L'**objectif** de ce projet est de combiner ces deux mécanismes afin de mettre en place une infrastructure web hautement disponible, comparable à celles déployées en entreprise. L'ensemble de l'infrastructure est réalisé sur VMware Workstation, à partir d'un schéma d'architecture fourni.

1.2 Objectifs pédagogiques

> **À l'issue de ce projet, je dois être capable de :**
>
> ✓ Maîtriser la configuration combinée de HAProxy et Heartbeat en cluster actif/passif ;
>
> ✓ Comprendre les mécanismes de bascule automatique (failover) d'une VIP entre deux nœuds ;
>
> ✓ Valider le fonctionnement par des tests de tolérance aux pannes documentés ;
>
> ✓ Produire une documentation technique structurée et professionnelle.

1.3 Enjeux de la haute disponibilité

La haute disponibilité (HA) répond à quatre grands enjeux.

Enjeu économique

Chaque minute d'arrêt (downtime) peut coûter des milliers d'euros.

- **Perte de chiffre d'affaires** : pour un site e-commerce, un arrêt de dix minutes en pleine période de soldes a des conséquences importantes.
- **Coût d'intervention** : mobiliser des experts en urgence, la nuit ou le week-end, est coûteux.
- **Pénalités contractuelles (SLA)** : un prestataire qui ne respecte pas ses engagements de service doit indemniser ses clients.

Enjeu d'image et de confiance

La réputation d'une entreprise repose en partie sur la fiabilité de ses outils numériques.

- **Crédibilité** : un service souvent « en maintenance » ou inaccessible renvoie une image peu professionnelle.
- **Fidélisation** : un utilisateur confronté à une erreur 500 se tournera vers un concurrent disponible.

- **Productivité interne** : lorsque les outils de travail tombent en panne, les employés perdent en efficacité et en motivation.

Enjeu technique : résilience et redondance

C'est l'enjeu principal de ce projet avec HAProxy et Heartbeat : éliminer le **SPOF** (Single Point of Failure), c'est-à-dire le point unique de défaillance.

- **Redondance matérielle** : disposer de plusieurs serveurs, alimentations et commutateurs.
- **Bascule automatique (failover)** : le système détecte une panne et bascule seul vers le secours, sans intervention humaine.
- **Maintenance transparente** : mettre à jour un serveur pendant que l'autre assure tout le trafic, sans « coupure pour maintenance ».

Enjeu de sécurité et d'intégrité

La haute disponibilité contribue également à la protection contre les menaces externes.

- **Résistance aux attaques** : un cluster de répartiteurs de charge absorbe plus facilement une attaque par déni de service (DDoS) qu'un serveur isolé.
- **Protection des données** : la HA s'accompagne souvent d'une réplication des données ; si un disque tombe en panne, l'information reste disponible ailleurs.

2. Présentation de l'architecture

L'infrastructure déployée est composée de cinq machines virtuelles réparties sur deux réseaux distincts : un réseau **WAN** (côté client) et un réseau **LAN** (côté serveurs).

2.1 Schéma d'architecture

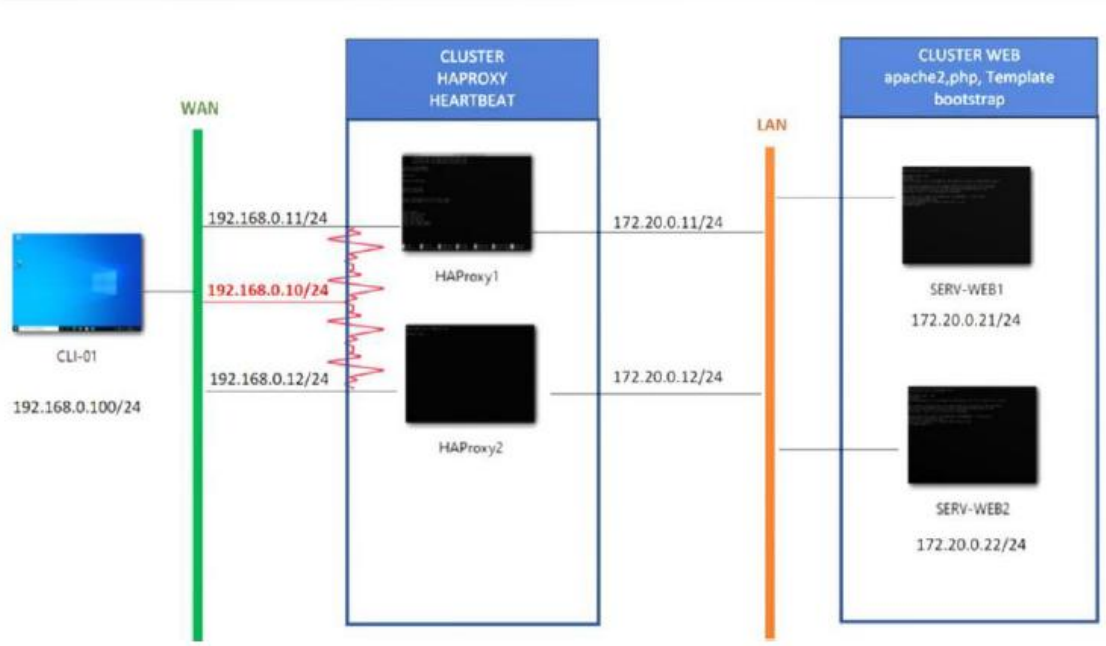


Figure 1 – *Schéma d'architecture du cluster*

2.2 Tableau d'adressage

VM	Rôle	IP WAN (eth0)	IP LAN (eth1)	RAM / vCPU
CLI-01	Client de test	192.168.0.100/24	—	1 Go / 1
HAProxy1	Load balancer primaire	192.168.0.11/24	172.20.0.11/24	1 Go / 1
HAProxy2	Load balancer secondaire	192.168.0.12/24	172.20.0.12/24	1 Go / 1
SERV-WEB1	Serveur web (Apache2/PHP)	—	172.20.0.21/24	1 Go / 1
SERV-WEB2	Serveur web (Apache2/PHP)	—	172.20.0.22/24	1 Go / 1

Adresse IP Virtuelle (VIP)
192.168.0.10/24 — adresse flottante gérée par Heartbeat côté WAN
Cette adresse est celle que le client CLI-01 utilisera pour accéder au service web.
Elle bascule automatiquement de HAProxy1 vers HAProxy2 en cas de panne.

Figure 2 – *Tableau d'adressage des machines virtuelles et de la VIP*

2.3 Plan d'adressage réseau

Deux réseaux VMware distincts (VMnet) sont créés :

Réseau	Sous-réseau	Type VMnet	Machines connectées
WAN	192.168.0.0/24	VMnet (Host-only)	CLI-01, HAProxy1, HAProxy2
LAN	172.20.0.0/24	VMnet (Host-only)	HAProxy1, HAProxy2, SERV-WEB1, SERV-WEB2

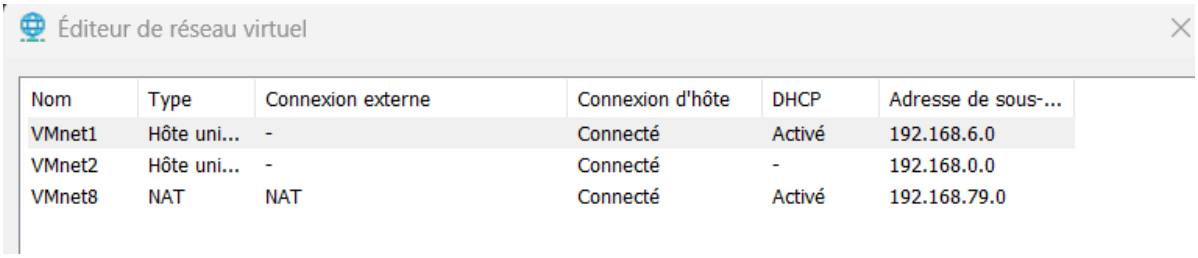
***Figure 3** – Plan d'adressage des réseaux WAN et LAN*

3. Réalisation étape par étape

Étape 1 – Préparation de l'environnement VMware

Création des réseaux virtuels WAN et LAN

Les deux réseaux virtuels sont créés depuis l'éditeur de réseau virtuel de VMware Workstation.



Nom	Type	Connexion externe	Connexion d'hôte	DHCP	Adresse de sous-...
VMnet1	Hôte uni...	-	Connecté	Activé	192.168.6.0
VMnet2	Hôte uni...	-	Connecté	-	192.168.0.0
VMnet8	NAT	NAT	Connecté	Activé	192.168.79.0

***Figure 4** – Éditeur de réseau virtuel VMware*

Création des cinq machines virtuelles

Conformément au tableau d'adressage, le projet comporte cinq machines virtuelles :

- deux répartiteurs de charge : **HAProxy1** et **HAProxy2** ;
- deux serveurs web : **SERV-WEB1** et **SERV-WEB2** ;
- un client Windows **CLI-01**, utilisé pour les tests.

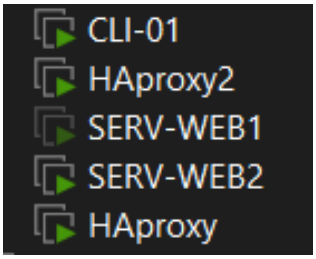
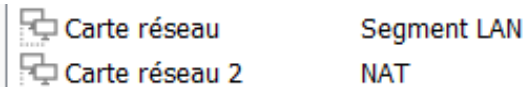


***Figure 5** – Machines virtuelles du projet*

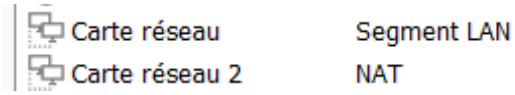
Configuration des interfaces réseau

Chaque machine virtuelle est rattachée aux segments réseau correspondant à son rôle :

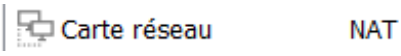
SERV-WEB1



SERV-WEB2



CLI-01



HAProxy1



HAProxy2

Figure 6 – *Configuration des cartes réseau de chaque VM*

Vérification de la connectivité

Avant de mettre en place les services de haute disponibilité, des tests de connectivité confirment que toutes les machines communiquent correctement. La commande utilisée est :

```
ping -c 4 <IP_CIBLE>
```

HAProxy1 → SERV-WEB1

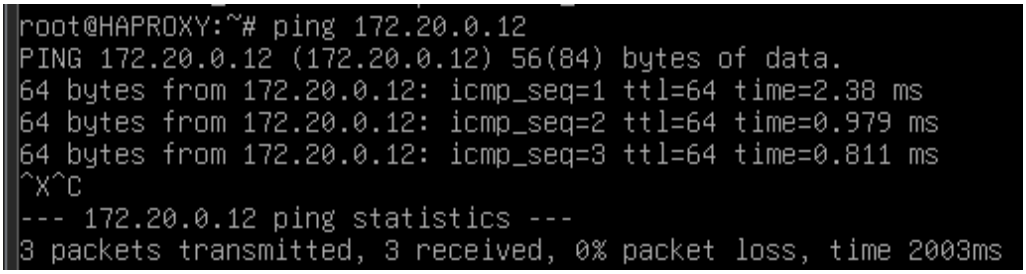


Figure 7 – *Ping de HAProxy1 vers SERV-WEB1*

HAProxy1 → SERV-WEB2

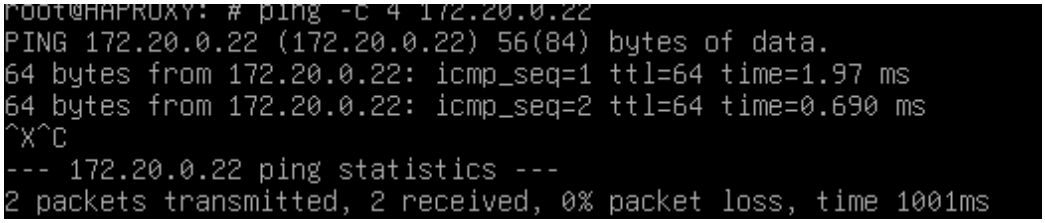


Figure 8 – *Ping de HAProxy1 vers SERV-WEB2*

HAProxy1 → HAProxy2

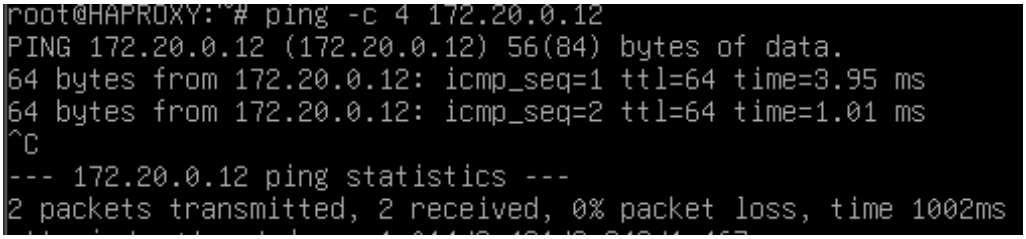


Figure 9 – *Ping de HAProxy1 vers HAProxy2*

CLI-01 → VIP du cluster

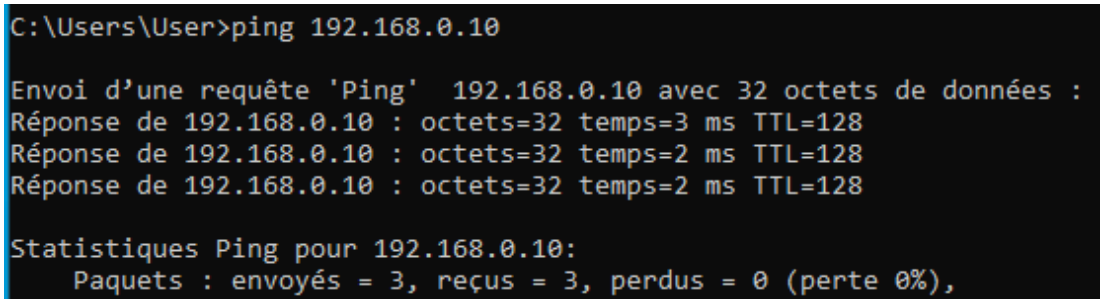


Figure 10 – *Ping de CLI-01 vers la VIP 192.168.0.10*

Étape 2 – Déploiement des serveurs web

Installation d'Apache2 et PHP

Les paquets nécessaires sont installés sur SERV-WEB1 et SERV-WEB2 :

```
apt update

apt install apache2 wget unzip -y
```

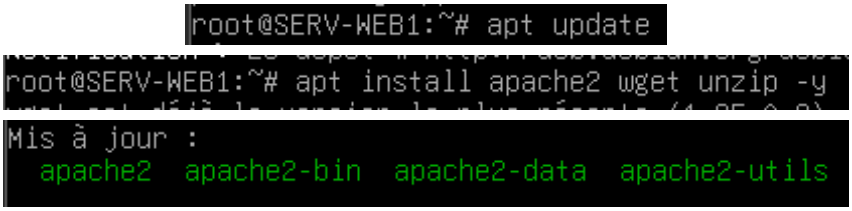


Figure 11 – *Installation d'Apache2 sur le serveur web*

Déploiement du site « The Grill »

On se place dans le répertoire racine du serveur web (`cd` signifie *change directory*) :

```
cd /var/www/html
```



Le modèle de site est ensuite téléchargé puis décompressé :



Figure 12 – *Téléchargement et décompression du site*

Modification du DocumentRoot d'Apache2

On édite le fichier de configuration du site par défaut :

```
nano /etc/apache2/sites-available/000-default.conf
```

```
root@SERV-WEB1:~# nano /etc/apache2/sites-available/000-default.conf
```

```
  GNU nano 8.4                                  /etc/apache2/sites-available/000-default.conf
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

Figure 13 – *Fichier 000-default.conf*

On ajoute le nom du dossier `thegrill-master` à la suite de `/var/www/html` dans la directive DocumentRoot :

```
DocumentRoot /var/www/html
```

```
root@SERV-WEB1:~# nano /var/www/html/thegrill-master/index.html
```

Figure 14 – *DocumentRoot modifié*

On enregistre avec **Ctrl + O**, puis on redémarre le service :

```
service apache2 restart
```

Personnalisation de la page d'accueil de chaque serveur

Afin d'identifier visuellement quel backend répond, le titre de la page est modifié sur chaque serveur :

```
nano /var/www/html/index.html
```

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8
<title>Apache2 Debian Default Page: SERV-WEB1</title>
<style type="text/css" media="screen">
```

Figure 15 – *Titre de la page sur SERV-WEB1*

```
<meta http-equiv="Content-Type" content="text/html; cha
<title>Apache2 Debian Default Page: SERV-WEB2</title>
```

Figure 16 – *Titre de la page sur SERV-WEB2*

Activation de Bootstrap via un CDN

Utiliser un **CDN** (Content Delivery Network) signifie que le serveur n'héberge pas lui-même les fichiers Bootstrap : c'est le navigateur du client qui les récupère sur un serveur externe optimisé. On ajoute la ligne suivante dans `index.html` sur **SERV-WEB1** et **SERV-WEB2** :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
```

Figure 17 – *Ajout de Bootstrap dans index.html*

Vérification de l'accès depuis les nœuds HAProxy

L'outil `curl` doit être installé au préalable :

```
apt install curl -y
```



Figure 18 – *Test d'accès au serveur web avec curl*

Étape 3 – Configuration du cluster HAProxy

Installation de HAProxy

HAProxy est installé sur **HAProxy1** et **HAProxy2** :

```
apt install haproxy
```

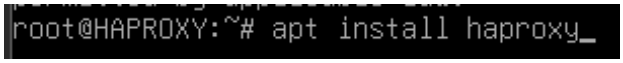


Figure 19 – *Installation de HAProxy*

Configuration identique sur les deux nœuds

> **Configuration attendue**
>
> - Frontend en écoute sur le **port 80** ;
> - Backend en **round-robin** vers SERV-WEB1 (172.20.0.21) et SERV-WEB2 (172.20.0.22) ;
> - **Health-check** activé sur chaque serveur.

On édite le fichier de configuration sur HAProxy1, puis on répète l'opération à l'identique sur HAProxy2 :

```
nano /etc/haproxy/haproxy.cfg
```



Figure 20 – *Configuration du frontend et du backend dans haproxy.cfg*

La commande `systemctl status haproxy` confirme que le service est actif (en vert) et activé au démarrage (enabled).

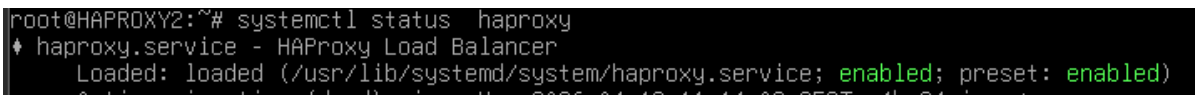


Figure 21 – *Statut du service HAProxy*

Activation de la page de statistiques

Les lignes suivantes sont ajoutées dans `/etc/haproxy/haproxy.cfg` sur **HAProxy1** et **HAProxy2** :

```
#Statistiques
stats enable
stats hide-version
stats refresh 30s
stats show-node
stats auth admin:iris2026
stats uri /statistiques
```

Figure 22 – *Section statistiques de haproxy.cfg*

Test du load balancing depuis CLI-01

Chaque nœud HAProxy est testé individuellement depuis le client CLI-01.

```
root@HAPROXY:~# systemctl start heartbeat
root@HAPROXY:~# systemctl stop heartbeat
```

```
C:\Users\User>ping 192.168.0.10 -t

Envoi d'une requête 'Ping'  192.168.0.10 avec 32 octets de données :
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=2 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps<1ms TTL=128
Délai d'attente de la demande dépassé.
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps<1ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
```

Figure 23 – *Tests depuis CLI-01*

Résultat : on observe une répartition équitable des requêtes entre les deux backends, conforme à l'algorithme **round-robin**.

Étape 4 – Configuration du cluster Heartbeat

Installation de Heartbeat

Heartbeat est installé sur **HAProxy1** (nœud primaire) et **HAProxy2** (nœud secondaire) :

```
apt install heartbeat wget unzip
```

```
root@HAPROXY2:~# apt install apache2 php heartbeat wget unzip_
```

Figure 24 – *Installation de Heartbeat*

L'installation crée le répertoire `/etc/ha.d`, dans lequel trois fichiers doivent être créés :

Fichier	Rôle
`ha.cf`	Fichier de configuration principal
`haresources`	Définition des ressources gérées par le cluster
`authkeys`	Informations d'authentification entre les nœuds

Fichier ha.cf

```
nano /etc/ha.d/ha.cf
```

On saisit la configuration ci-dessous, puis on enregistre avec **Ctrl + O** :

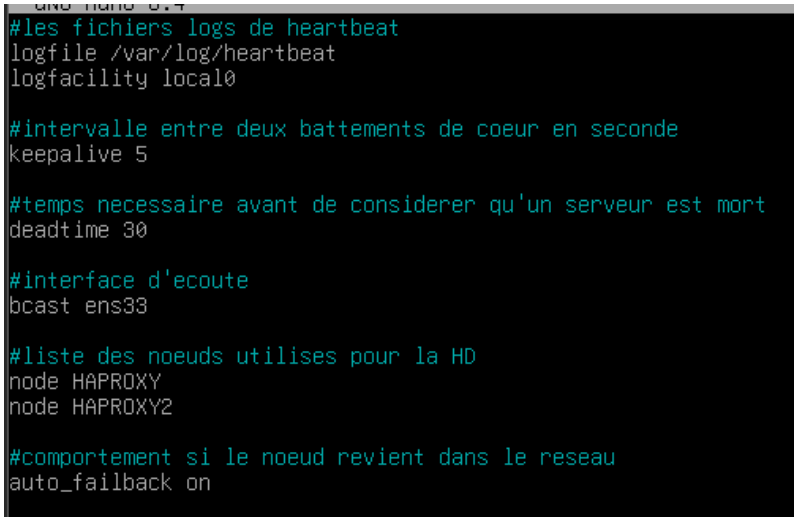


Figure 25 – *Fichier /etc/ha.d/ha.cf*

Fichier haresources

```
nano /etc/ha.d/haresources
```



Figure 26 – *Fichier /etc/ha.d/haresources*

Fichier authkeys

```
nano /etc/ha.d/authkeys
```



Figure 27 – *Fichier /etc/ha.d/authkeys*

Définition de la VIP comme ressource Heartbeat

La configuration doit être **strictement identique** sur les deux nœuds. Le fichier `/etc/ha.d/haresources` contient :

```
HAPROXY1 IPaddr::192.168.0.10/24/ens37 haproxy
```



Figure 28 – *Déclaration de la VIP dans haresources*

Heartbeat crée ainsi la VIP sur l'interface **ens37** et démarre le service HAProxy. Le bon fonctionnement est vérifié avec :

```
journalctl -u heartbeat
```

```
avril 08 15:31:46 HAPROXY systemd[1]: Started heartbeat.service - Heartbeat High Availability Cluster Communication and Membership.
```

Figure 29 – *Journal du service Heartbeat*

Vérification de la VIP sur le nœud maître

La commande `ip addr show` exécutée sur HAProxy1 doit faire apparaître deux adresses sur l'interface WAN :

- l'IP physique **192.168.0.11**, propre à la machine ;
- l'IP virtuelle (VIP) **192.168.0.10**.

```
root@HAPROXY:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:91:b5:17 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2991b517
    inet 172.20.0.11/24 brd 172.20.0.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe91:b517/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:91:b5:21 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c2991b521
    inet 192.168.0.11/24 brd 192.168.0.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet 192.168.0.10/24 brd 192.168.0.255 scope global secondary ens37:1
       valid_lft forever preferred_lft forever
```

```
root@HAPROXY2:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:aa:a4:ee brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29aaa4ee
    inet 172.20.0.12/24 brd 172.20.0.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feaa:a4ee/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:aa:a4:f8 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c29aaa4f8
    inet 192.168.0.12/24 brd 192.168.0.255 scope global ens37
       valid_lft forever preferred_lft forever
```

Figure 30 – *La VIP est bien portée par HAProxy1*

Étape 5 – Tests de bascule et validation

Une série de quatre tests est réalisée depuis le client **CLI-01** afin de valider le fonctionnement du cluster.

Test 1 – Répartition de charge (round-robin)

> **Objectif**
> Accéder à http://192.168.0.10 et vérifier l'alternance des réponses entre SERV-WEB1 et SERV-WEB2.

Depuis un navigateur sur CLI-01, on accède à `http://192.168.0.10`. À chaque rafraîchissement, la page affiche alternativement « THE GRILL 1 » et « THE GRILL 2 », ce qui prouve que les deux serveurs répondent tour à tour.

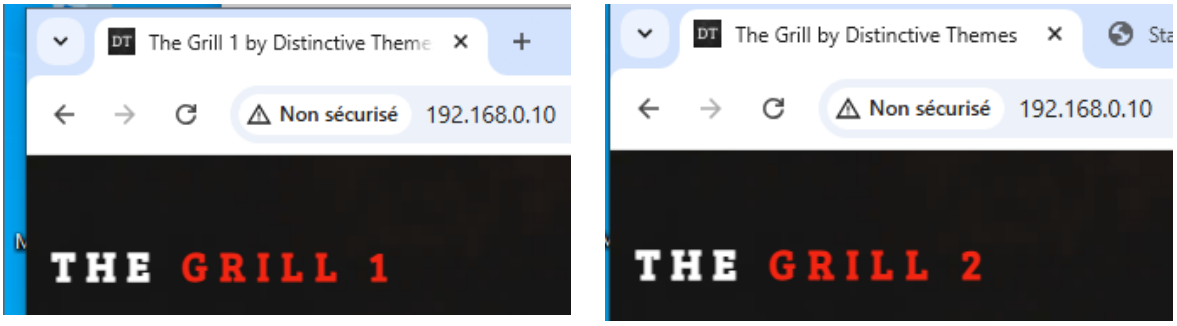


Figure 31 – Réponses alternées de SERV-WEB1 et SERV-WEB2

La page de statistiques HAProxy (`http://192.168.0.11/statistiques`) confirme la répartition des sessions entre les deux backends.

WEB_CLUSTER_SIO1_jv

	Queue			Session rate			Sessions					
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last
Frontend				1	2	-	2	3	262 117	22		
SERV-WEB1	0	0	-	0	1		0	1	-	3	3	4s
SERV-WEB2	0	0	-	1	1		0	1	-	3	3	2s
Backend	0	0		1	1		0	1	26 212	6	6	0s

Figure 32 – Sessions réparties dans les statistiques HAProxy

Résultat : ✓ conforme

Test 2 – Panne du nœud HAProxy1 (failover)

> **Objectif**
> Simuler une panne de HAProxy1, constater la bascule de la VIP vers HAProxy2 et vérifier que le service reste accessible depuis CLI-01.

1. Sur HAProxy1, on arrête le service Heartbeat :

```
systemctl stop heartbeat
```

```
root@HAPROXY:~# systemctl start heartbeat
root@HAPROXY:~# systemctl stop heartbeat
```

2. Sur CLI-01, un ping continu est lancé vers la VIP :

```
ping 192.168.0.10 -t
```

On observe une très brève interruption, puis la reprise immédiate des réponses : c'est la bascule de la VIP de HAProxy1 vers HAProxy2.

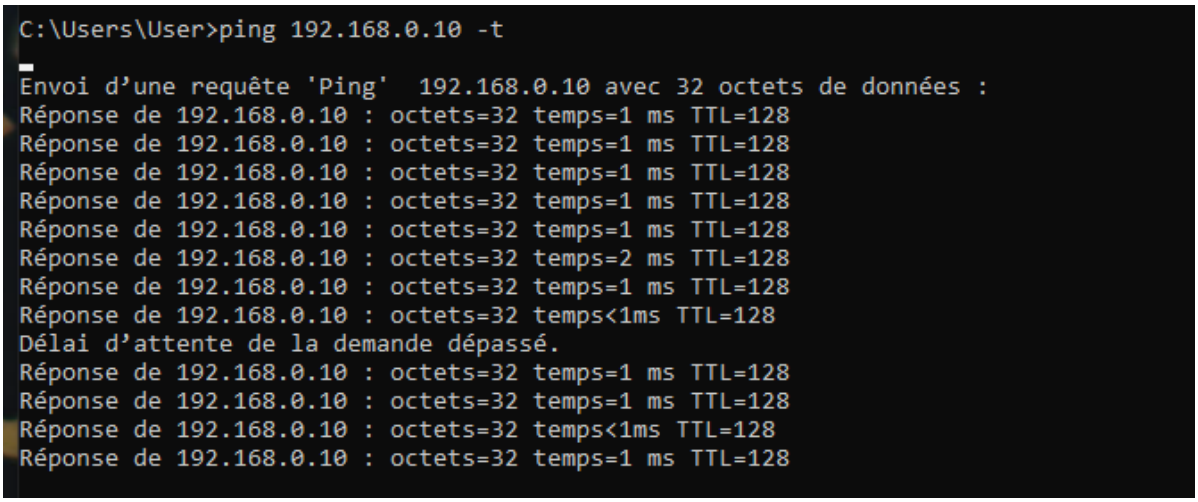


***Figure 33** – Ping continu pendant la bascule vers HAProxy2*

Résultat : ✓ conforme

Test 3 – Panne d'un serveur backend (health-check)

> **Objectif**
> Arrêter Apache2 sur SERV-WEB1 et vérifier que HAProxy redirige automatiquement tout le trafic vers SERV-WEB2.

Ce test valide le **health-check** : HAProxy doit détecter qu'un serveur ne répond plus et cesser de lui envoyer des utilisateurs. On simule pour cela une panne logicielle sur SERV-WEB1.

1. Sur SERV-WEB1, on arrête le service Apache :

```
systemctl stop apache2
```



2. Sur la page de statistiques, SERV-WEB1 passe en rouge (état DOWN).

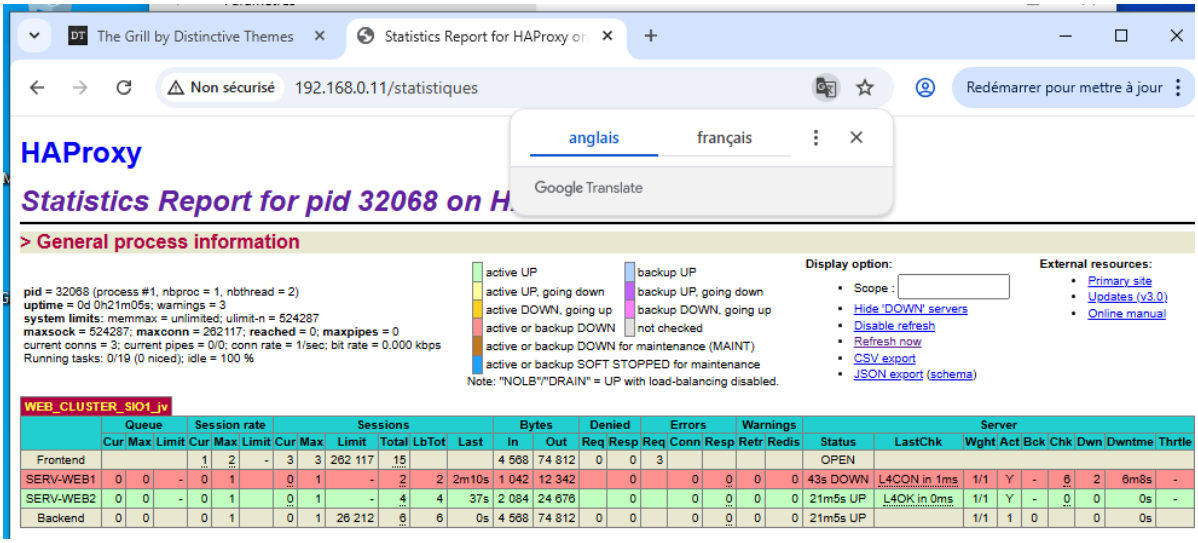


Figure 34 – SERV-WEB1 détecté hors service par HAProxy

3. En accédant à `http://192.168.0.10`, seule la page « THE GRILL 2 » s'affiche désormais : tout le trafic est servi par SERV-WEB2.



Figure 35 – Seul SERV-WEB2 répond

Résultat : ✓ conforme

Test 4 – Retour du nœud primaire (failback)

> **Objectif**
>
> Remettre en service HAProxy1 et vérifier le retour à la normale : le nœud primaire reprend la VIP.

HAProxy1 étant déclaré nœud prioritaire dans `haresources`, et l'option `auto_failback on` étant activée dans `ha.cf`, il réclame ses ressources dès son retour. HAProxy2 lui rend alors automatiquement la VIP.

1. Avant le redémarrage, `ip a` sur HAProxy2 montre que ce nœud porte toutes les ressources, dont la VIP.

```
root@HAPROXY2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:aa:a4:ee brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29aaa4ee
    inet 172.20.0.12/24 brd 172.20.0.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feaa:a4ee/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:aa:a4:f8 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c29aaa4f8
    inet 192.168.0.12/24 brd 192.168.0.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet 192.168.0.10/24 brd 192.168.0.255 scope global secondary ens37:1
       valid_lft forever preferred_lft forever
```

Figure 36 – *La VIP est portée par HAProxy2*

2. Un ping continu est lancé depuis CLI-01 vers 192.168.0.10, puis on relance Heartbeat sur HAProxy1 :

```
systemctl start heartbeat
```

Une courte interruption du ping est observée au moment où HAProxy1 reprend ses ressources.

```
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
Délai d'attente de la demande dépassé.
Réponse de 192.168.0.10 : octets=32 temps=1 ms TTL=128
```

Figure 37 – *Ping pendant le retour de HAProxy1*

3. La commande `ip addr show` sur HAProxy1 confirme que la VIP est de retour sur le nœud principal.

```
root@HAPROXY:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:91:b5:17 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2991b517
    inet 172.20.0.11/24 brd 172.20.0.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe91:b517/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:91:b5:21 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c2991b521
    inet 192.168.0.11/24 brd 192.168.0.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet 192.168.0.10/24 brd 192.168.0.255 scope global secondary ens37:1
       valid_lft forever preferred_lft forever
```

Figure 38 – *La VIP est de nouveau portée par HAProxy1*

Résultat : ✓ conforme

4. Bilan

4.1 Synthèse des tests

Test	Scénario	Résultat attendu	Statut
1	Accès à la VIP	Alternance SERV-WEB1 / SERV-WEB2	✓ Validé
2	Arrêt de HAProxy1	Bascule de la VIP vers HAProxy2	✓ Validé
3	Arrêt d'Apache2 sur SERV-WEB1	Trafic redirigé vers SERV-WEB2	✓ Validé
4	Redémarrage de HAProxy1	Reprise de la VIP par le nœud primaire	✓ Validé

4.2 Conclusion

Ce projet m'a permis de mettre en place une infrastructure web hautement disponible combinant deux niveaux de redondance :

- la **répartition de charge** assurée par HAProxy, avec détection automatique des serveurs web défaillants (health-check) ;
- la **continuité de service** du point d'entrée assurée par Heartbeat, grâce à une VIP qui bascule automatiquement entre les deux répartiteurs.

Les quatre tests réalisés confirment que le cluster fonctionne en mode **actif/passif** (nœud maître / nœud secondaire) : il supporte la panne d'un répartiteur comme celle d'un serveur web, et revient automatiquement à sa configuration nominale sans intervention manuelle. Les seules interruptions observées sont de très courtes pertes de paquets au moment des bascules.